

GRAPH

PROBLEM 9

QUESTION:

DS	Session	Graph	Question Information	Level 1	Challenge 89
Question description A game consists of n rooms and m teleporters. At the beginning of each day, you start in room 1 and you have to reach room n. You can use each teleporter at most once during the game. How many days can you play if you choose your routes optimally? Constraints $2 \leq n \leq 500$ $1 \leq m \leq 1000$ $1 \leq a, b \leq n$ Input The first input line has two integers n and m: the number of rooms and teleporters. The rooms are numbered $1, 2, \dots, n$. After this, there are m lines describing the teleporters. Each line has two integers a and b: there is a teleporter from room a to room b. There are no two teleporters whose starting and ending room are the same. Output First print an integer k: the maximum number of days you can play the game. Then, print k route descriptions according to the example. You can print any valid solution.					
▼ Logical Test Cases Test Case 1 INPUT (STDIN) <pre>6 7 1 2 1 3 2 6 3 4 3 5 4 6 5 6</pre> EXPECTED OUTPUT <pre>2 4 1 3 5 6 3 1 2 6</pre> Test Case 2 INPUT (STDIN) <pre>6 7 1 4 3 5 4 6 5 6 2 2 1 3 2 6</pre> EXPECTED OUTPUT <pre>2 4 1 3 5 6 3 1 4 6</pre>					
▼ Mandatory Test Cases Test Case 1 KEYWORD <pre>int bfs(int n,int s,int t)</pre>					
▼ Complexity Test Cases Test Case 1 CYCLOMATIC COMPLEXITY 7 Test Case 2 TOKEN COUNT 920 Test Case 3 NLOC 110					

SOURCE CODE:

```

#include <stdio.h>
#include <stdlib.h>
#include <stdbool.h>

#define MAX_N 505
#define MAX_M 2005

typedef struct Pipe {
    int to, cap, flow, rev;
} Pipe;

Pipe pipes[2 * MAX_M];
int start_pipe[MAX_N], next_pipe[2 * MAX_M], pipe_counter = 0;

void add_game_pipe(int from, int to, int capacity) {
    pipes[pipe_counter] = (Pipe){to, capacity, 0, pipe_counter + 1};
    next_pipe[pipe_counter] = start_pipe[from];
    start_pipe[from] = pipe_counter++;

    pipes[pipe_counter] = (Pipe){from, 0, 0, pipe_counter - 1};
    next_pipe[pipe_counter] = start_pipe[to];
    start_pipe[to] = pipe_counter++;
}

int parent_pipe[MAX_N];

bool route_bfs(int total_rooms, int src, int sink) {
    for (int i = 1; i <= total_rooms; i++) parent_pipe[i] = -1;
    int queue[MAX_N], front = 0, rear = 0;
    queue[rear++] = src;
    parent_pipe[src] = -2;

    while (front < rear) {
        int node = queue[front++];
        if (node == sink) return true;
        for (int p = start_pipe[node]; p != -1; p = next_pipe[p]) {
            int nxt = pipes[p].to;
            if (parent_pipe[nxt] == -1 && pipes[p].cap > pipes[p].flow) {
                parent_pipe[nxt] = p;
                queue[rear++] = nxt;
            }
        }
    }
    return false;
}

int main() {
    int total_rooms, portals;
    if (scanf("%d %d", &total_rooms, &portals) != 2) return 0;

    for (int i = 1; i <= total_rooms; i++) start_pipe[i] = -1;
    for (int i = 0; i < portals; i++) {
        int a, b;
        scanf("%d %d", &a, &b);
        add_game_pipe(a, b, 1);
    }

    int viable_days = 0;
    while (route_bfs(total_rooms, 1, total_rooms)) {
        for (int v = total_rooms; v != 1; ) {
            int p = parent_pipe[v];
            pipes[p].flow += 1;
            pipes[pipes[p].rev].flow -= 1;
            v = pipes[pipes[p].rev].to;
        }
        viable_days++;
    }

    printf("%d\n", viable_days);
    for (int d = 0; d < viable_days; d++) {
        int path_nodes[MAX_N], path_sz = 0;
        int cur_room = 1;
        path_nodes[path_sz++] = cur_room;
        while (cur_room != total_rooms) {
            for (int p = start_pipe[cur_room]; p != -1; p = next_pipe[p]) {
                if (pipes[p].flow == 1) {
                    pipes[p].flow = 0;
                    cur_room = pipes[p].to;
                    path_nodes[path_sz++] = cur_room;
                    break;
                }
            }
        }
        printf("%d", path_sz);
        for (int i = 0; i < path_sz; i++) printf(" %d", path_nodes[i]);
        printf("\n");
    }
    return 0;
}

```

OUTPUT:

```
6 7
1 2
1 3
3 4
3 5
4 6
5 6
2 6
2
3 1 2 6
4 1 3 4 6
```

```
6 7
1 4
3 5
4 6
5 6
2 2
1 3
2 6
2
4
1 3 5 6
3
1 4 6
```